Turbulence Model Calibration using 2D Backward Facing step

1. Introduction

This is my first attempt at optimizing a turbulence model. My main learning objectives for this project are as follows:

- Learn the basics of turbulence model calibration / optimisation
- Look at the different ways / algorithms used for this
- Create a framework that can be applied to other cases

Sidenote, I interchange between turbulence model calibration and optimisation; I still haven't decided which word i like more.

2. Project file structure and Script overview

The project is adapted off of the backward facing step case from the OpenFOAM tutorials directory. As such, the project root contains dirs from this tutorial case, such as `constant`, `system`, `0`, `postProcessing`, etc..

The grids used for the turbulence calibration are taken from the NASA Turbulence Modelling resource page for the same case [1]. The grids are located in the `NASA_Grids` directory. The grids are named in the format `backstep5_{GL}levdn.p{dim}fmt`, where *GL* is the grid level, with 0 being the finest; and *dim* is either *p2d* or *p3d* referring to 2D and 3D grids respectively.

`ValidationData` contains the experimental data under **driverDataFiles**. The data has been extracted into csv files, and located under the `backstep_csv` dir, with subdirs **RST.0** and **RST.6** referring to cases with **0 deg** and **6 deg** slant angle respectively. **Images** and **post_data** contain some postprocessing data of these cases run by me in OpenFOAM.

dataForOptLoop is a dir for convenience, containing the relevant csv files for the calibration. **cases** and **ax_result_data** contain the trial runs and results from ax (the optimizer, will be explained later)

As for scripts, the 3 allrun files (i.e **Allrun, Allrun_parallel, Allrun-w_pyfoam**) are used for running a simulation on the NASA Grids. **bayesOpt.py** implements the main optimisation framework, with variables and configurations located in **centralConfig.py**. **errorCalc.py** is the implementation of the RMSE (Root Mean Square Error) used as the objective in the optimisation. **runOpt.sh** is a script for executing the optimiser, once configurations are made. **Allclean** will prepare the project directories for a new optimiser session / run.

3. Framework Explanation

The first thing i did was do some research into the field of turbulence model calibration. From what I saw, the main approach seems to be to either use traditional optimisation algorithms, or neural networks. Here, I chose to go with the optimisation algorithm route, specifically Bayesian optimisation, primarily because its easier for me to implement and work with. Training a neural network and dealing with the validation overhead would probably be too much for my computational resource and time availability.

Python has several packages for implementing Bayesian optimisation (ex: ax-platform, GPyOpt, a package literally called bayesian-optimization,…). Out of these, I chose ax-platform (also referred to as ax) for the following reasons:

- The code to setup a problem seemed pretty straightforward and minimal
- It has an inbuilt feature for post processing the optimisation run

However, as I started with my project I encountered a very big drawback, lack of documentation. The official docs can give you a rough idea of the overall flow of ax, but it doesn't really explain a lot of concepts, and the source code in-line documentation was not too good either, making it somewhat of a pain to debug, and implement features that weren't fully explained in the docs.

The main script implementing the core definitions for the optimisation is `bayesOpt.py`. There are 3 big sections, the *Runner* and *ErrorMetric* class definition, and the **save_card** function definition.

The `Runner` class defines the functions needed to run a *trial* (i.e A simulation run with a particular combination of turbulence coefficients) and check its status. In order to run a trial, the following steps are taken:

1. A new dir is created under the `cases` dir, named `trial_{trial_index}`
2. Case setup files are copied into this dir, i.e `0`, `constant`, `system` and `dataForOptLoop`. Note that the mesh is also copied, under the `constant` dir
3. Turbulence coefficients are updated
4. The case is decomposed and simpleFoam is run in parallel through a pyFoam module
5. Velocity fields are sampled at pre-defined locations and moved to the `dataForOptLoop` folder on trial completion

A note on the mesh - The mesh from the NASA Turbulence resource consisted of raw grid points, and therefore some preprocessing had to be done to setup the necessary patch faces. The `Allrun` scripts can be used for this purpose.

Popen is used to run simpleFoam in a non-blocking manner.The `poll_trial` polls this instance to assess its status. A simple divergence check is also implemented: if the initial residuals from the final time step are greater than those of the first time step, the trial is considered to have diverged and the trial is ignored.

Next, let's look at the `ErrorMetric` class. This defines the objective that will be minimized, i.e the Root Mean Square Error (RMSE) between the velocity from the simulation and experimental velocity data. The simulation writes out velocity data at pre-defined positions to csv files, which are moved to the `dataForOptLoop` dir, and the error is then calculated. The error is a weighted sum of the RMSE near and away from the wall. By default, I assigned a weight of 1.5 to the near wall error, and 0.5 to that of the velocity away from the wall.

The error is returned as a tuple, (`0`, {`RMSE`}). Ax requires this class to return data in the form of ({`progression`}, {`objective`}), and I'll be completely honest, I'm not sure what the 'progression' value is meant to be. The doc doesn't really explain what this value means, only that it can be set to 0, so its set to 0.

The setup variables for `bayesOpt.py` can be configured in the `centralControl.py` script. It's necessary to run this script before `bayesOpt.py`, if changes are made to the `controlDict` or `decomposeParDict` variables (i.e `MAX_ITER`, `write_control`, `NPROC`). The `runOpt.sh` script first cleans the project root, runs `centralControl.py` and then runs `bayesOpt.py`, so it's better to use this script for starting an optimisation run to ensure that your changes are preserved in the run.

3.1. Note on time between polls

One of the variables available in `centralControl.py` is `TIME_BW_POLLS`. When ax runs a trial, it regularly checks whether the trial is still running or completed. In my script, this check is done by using the `poll()` method on the trial subprocess. The time between consecutive checks (or 'polls') follows the formula:

$$(1.5^n) * \texttt{TIME\_BW\_POLLS} \tag{1}$$

Where $n \geq 0$ represents the n^{th} poll, with $n = 0$ representing the first poll, $n = 1$ the second, and so on.

A trial is marked as completed **only if** it has been polled after completion and `TrialStatus.COMPLETED` is returned. This means that even if the trial (i.e here, the simulation) has completed, ax will not move on to the next trial unless the current one has been polled. The time spent between case completion and the subsequent poll can be expressed as

$$\text{wait\_time} = (1.5^n) * \texttt{TIME\_BW\_POLLS} - \text{case\_completion\_time} \tag{2}$$

To minimize the overall duration of the optimization run, we'd want to minimize this `wait_time`. The logical first consideration would be to want a $0s$ wait time, i.e

$$(1.5^n) * \texttt{TIME\_BW\_POLLS} = \text{case\_completion\_time} \tag{3}$$

If we assume that the computational overhead of making a poll is negligible, we can ignore n as a variable, and set it to 0 for convenience. We then have,

$$\texttt{TIME\_BW\_POLLS} = \text{case\_completion\_time} \tag{4}$$

So if we know how long our base case takes to complete, we can assign the same value to `TIME_BW_POLLS` and we theoretically should have no time wasted between case completion and polling. But this is probably **not a good idea**.

Let's assume we're executing an optimization run with `case_completion_time` = 100s, and `TIME_BW_POLLS` = 100s. If a trial completes in 101s, we'll be wasting 49s waiting for the next poll to happen and mark the trial as complete ($1.5 * 100 = 150 \Rightarrow 150 - 101 = 49s$ wait time). Therefore, if a trial completes even a little bit later than the prescribed poll time, a lot of time will be wasted. In this example, it's not a big issue, but if you're working on an HPC system, with cases that take hours to complete, the wasted time quickly adds up.

A better option is to find out or make an educated guess on the variance of `case_completion_time` throughout the optimization run. If we assume a $\pm 10\%$ variation, using Equation 2 we get:

$$\texttt{TIME\_BW\_POLLS} = 1.2 * \text{case\_completion\_time} \tag{5}$$

Going back to the earlier example, if a trial completes in 101s, we would now wait for only 20s for the poll. Of course, with more tighter tolerances this wait time can be further reduced.

Therefore, it is **very important to assign an appropriate value to the `TIME_BW_POLLS` variable** in `centralControl.py`, in order to minimize overall optimization run duration. A little bit of time spent deciding this variable can produce a noticeable reduction on overall computation time.

3.2. Turbulence model overview

Now, some words on the turbulence model. OpenFOAM already has a version of the backward facing step case in its tutorials (hereafter referred to as *base_OF*), using the k-ω SST model, so I just used the same thing. Now, this model has many coefficients in its formulation, so I looked into the literature to see which coefficients had the greatest impact on model predictions. From what i read [2], [3], the coefficients `a1` and β^* were the most important.

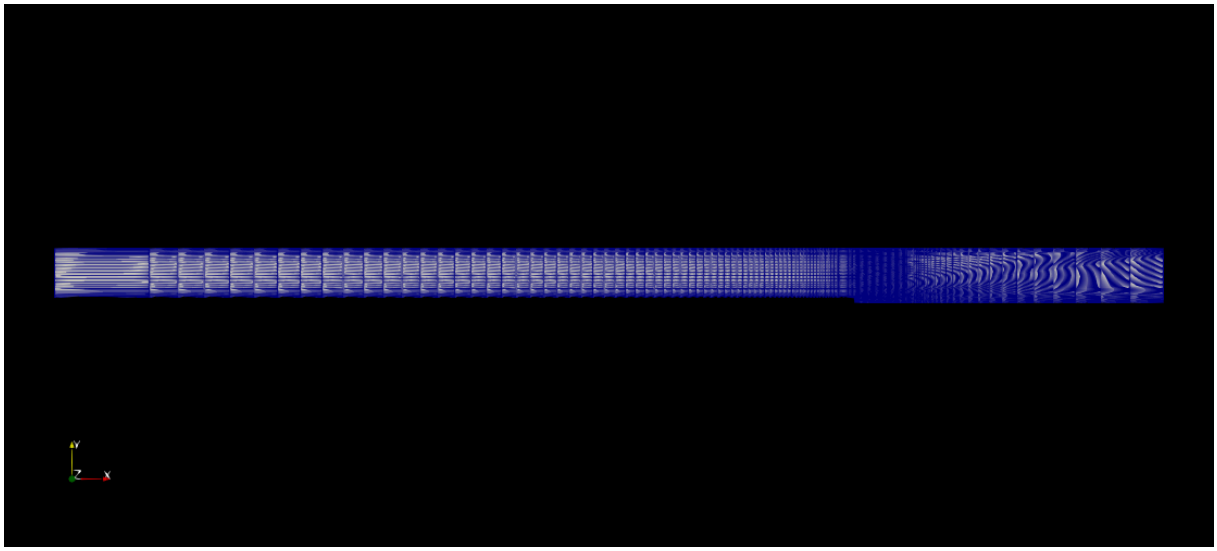
`a1` is the factor of proportionality between turbulent shear stress and turbulent kinetic energy (TKE) in the boundary layer. Therefore, simulated flow in boundary layers is strongly influenced by `a1`. β^* appears in the equations for TKE production and dissipation, and also the turbulence production limiter. These two equations influence both the boundary layer, and also wake behavior. Therefore, by modifying `a1` and β^*, we are able to influence the most important regions (i.e boundary layer and wake) in the fluid flow.

4. Results and Discussion

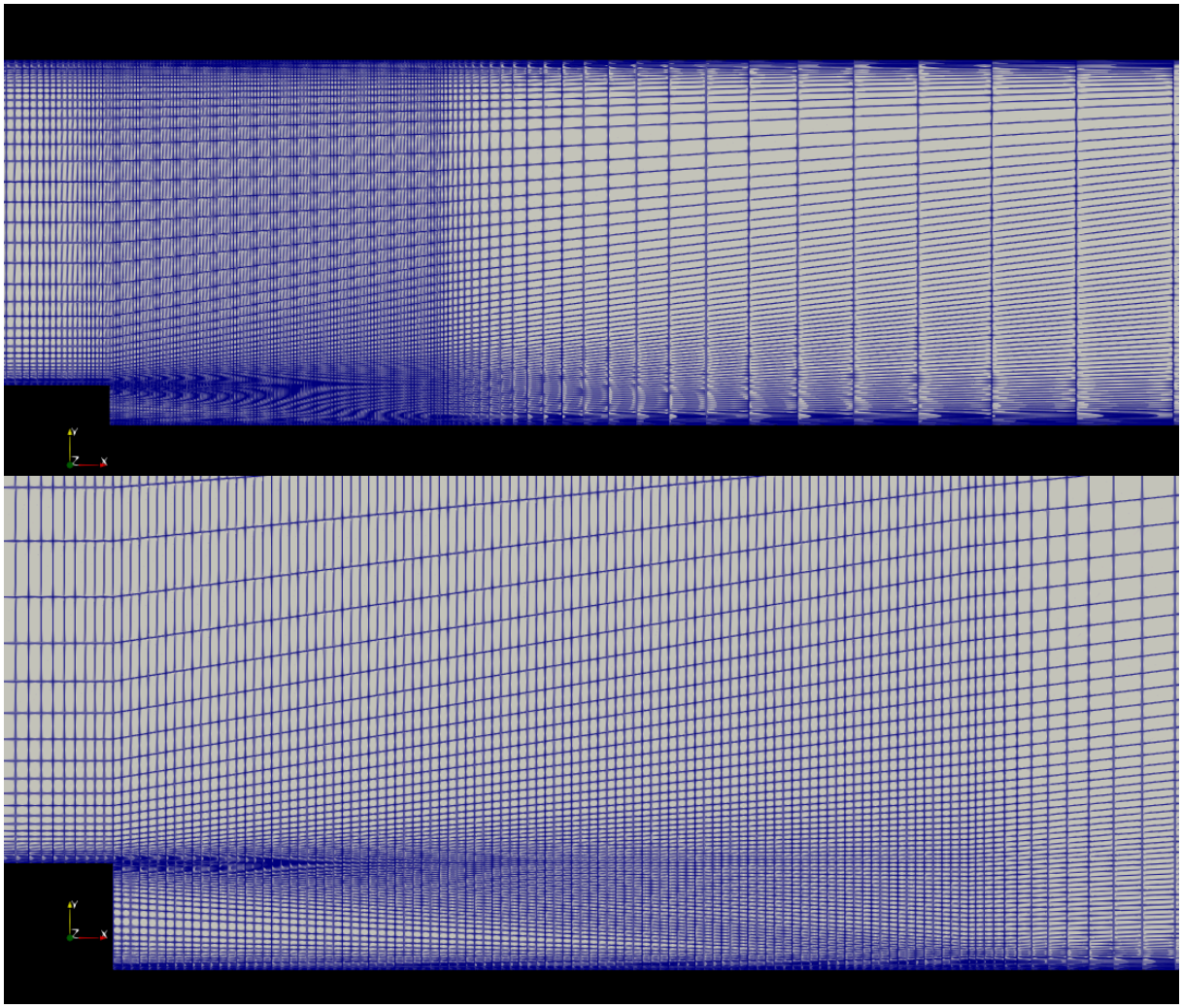
I've done two sets of simulations. I first worked with the grids from the NASA Turbulence page [1], specifically Grid Level 1, and then used the same workflow on the *base_OF* case. Both cases were run for 15 trials each, with 4000 openFOAM iterations for the NASA Grid and 2000 for the *base_OF* grid. `a1` was varied from 0.155 to 0.465 and β^* varied from 0.045 to 0.135. Velocity was sampled at ${}^{x}/_{H} = 1, 4, 6, 10$ ($H \Rightarrow$ Step Height). As mentioned in Section 2, the total error is a weighted sum of the RMSE near (`w1` $= 1.5$) and away (`w2` $= 0.5$) from the wall. Near wall is defined where ${}^{y}/_{H} \leq 1.2$, and anything beyond is seen as away from the wall.

4.1. Grids

First, let's look at the grids:

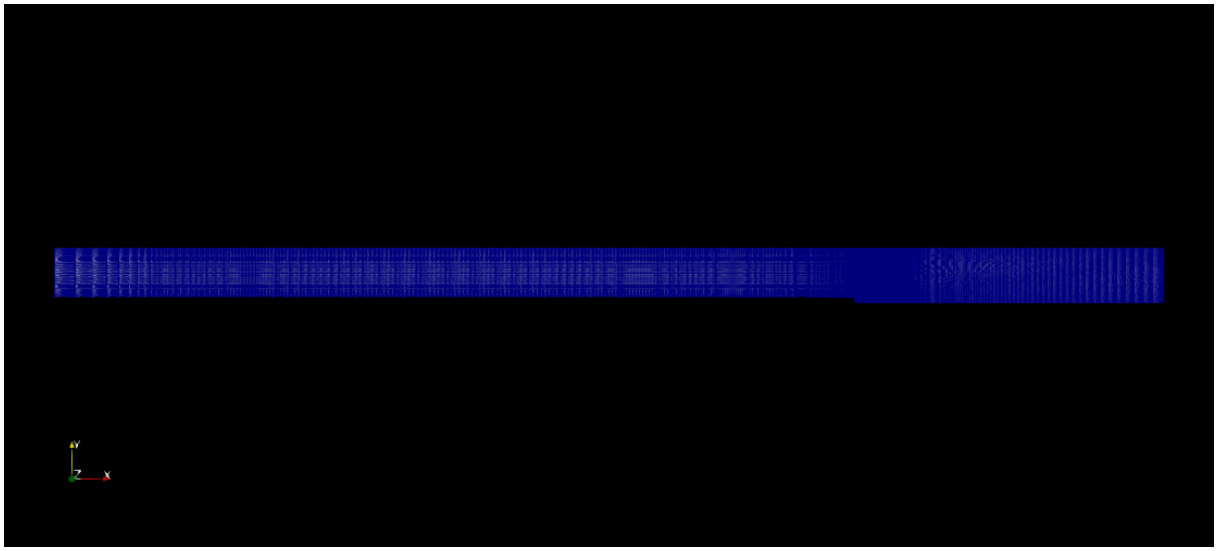


(a) Overall grid

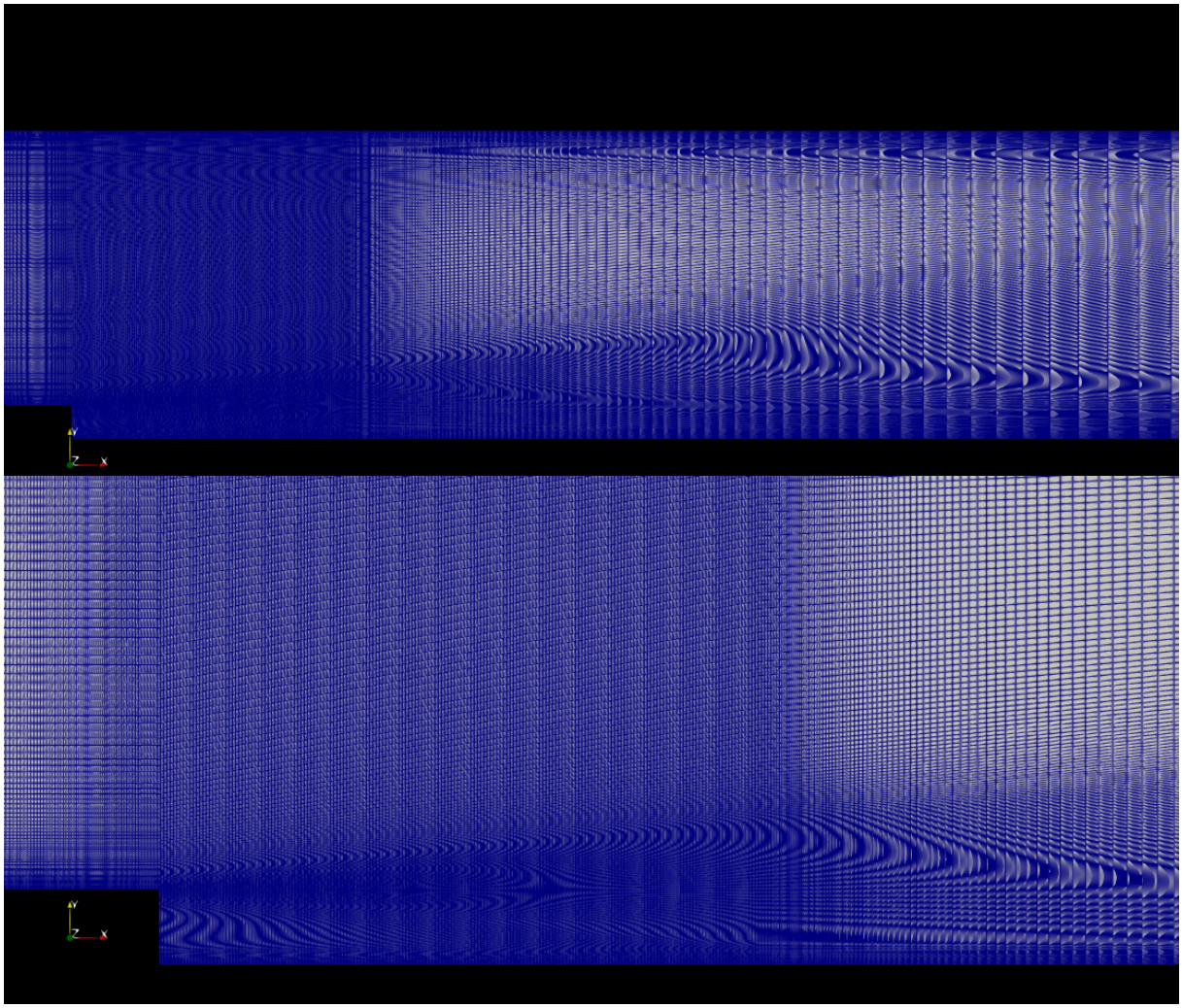


(c) Zoomed view of the step

Figure 1: *base_OF* blockMesh Grid



(a) Overall grid.



(c) Zoomed view of the step

Figure 2: NASA Grid, leve 1

Figure 1 and Figure 2 show the grids used. It is quite clear from the figures that the NASA grid is much more refined compared to the blockMesh. The blockMesh has 20,540 cells, whereas the NASA grid has 319,488 cells (around 16 times as many cells!), average y^+ for the lower wall in the blockMesh is 1.3, and 20 for the NASA Grid. Since we use the k-ω SST model, it should be able to handle both these grids.

Initially, I'd expected the NASA Grid to give me a better result than the blockMesh because of the refinement near the step, which should help in better resolving the separated region after the step, but due to the y+ being near the buffer region, I wasn't too sure what to expect. Furthermore, since the NASA grids were globally refined, there is a lot of unnecessary refinement throughout the domain, which would increase computational time.

From Figure 1(a),the high aspect ratio cells near the inlet and downstream of the step, which should be adequate in modelling the flow there, as you don't expect any deviation / changes in the flow near the inlet, and downstream of the step, you'd expect the flow to reattach and be steady. The *base_OF* blockMesh grid is probably more practical for a design use case, whereas the NASA Grid is better suited for a systematic validation run with uniform grid refinement (I would also probably try to get the y^+ out of the buffer zone though)

4.2. Calibration results

Now, lets look at the results:

Case	**Total RMSE (ms^{-1})**	**Error reduction compared to baseline**	**Calibrated Coefficients**	**Approx case completion time (s)**
NASA Grid 1	3.53	26%	$\texttt{a1} = 0.411$ $\beta^* = 0.045$	1500
base_OF	0.51	25%	$\texttt{a1} = 0.410$ $\beta^* = 0.078$	20

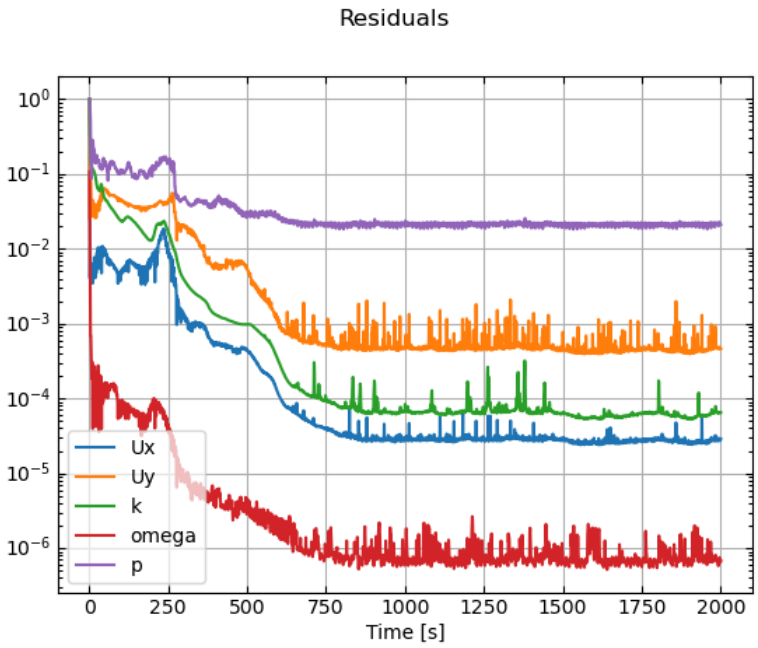


Figure 3: Residuals of the best *base_OF* case

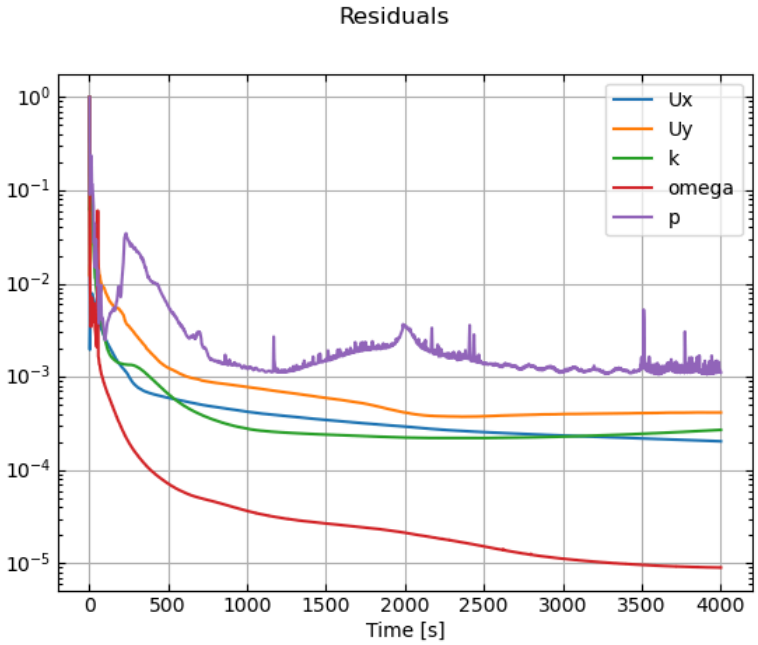


Figure 4: Residuals of the best NASA Grid level 1 case

From the table, it is clear that the *base_OF* case gets a better fit to the experimental data as compared to the NASA Grid. Looking at the residuals, we see that the NASA Grid has not yet converged at 4000 iterations, and could probably use a few thousand iterations more. However, at this point, you probably want to ask yourself : Is it even worth it to continue with the NASA Grid? Even if it has the potential for a better answer, is that worth the time spent? The answer

to this question is heavily case dependent, but from what I understand about optimisation, it is generally better to get a "good enough" answer within a short duration, rather than spend hours searching for "the best" answer. If this were an industrial setting where I wanted to use the optimised turbulence model for other calculations, I'd go with the results from the *base_OF* case and also use it for any further turbulence optimisation testing.

4.3. Velocity comparison

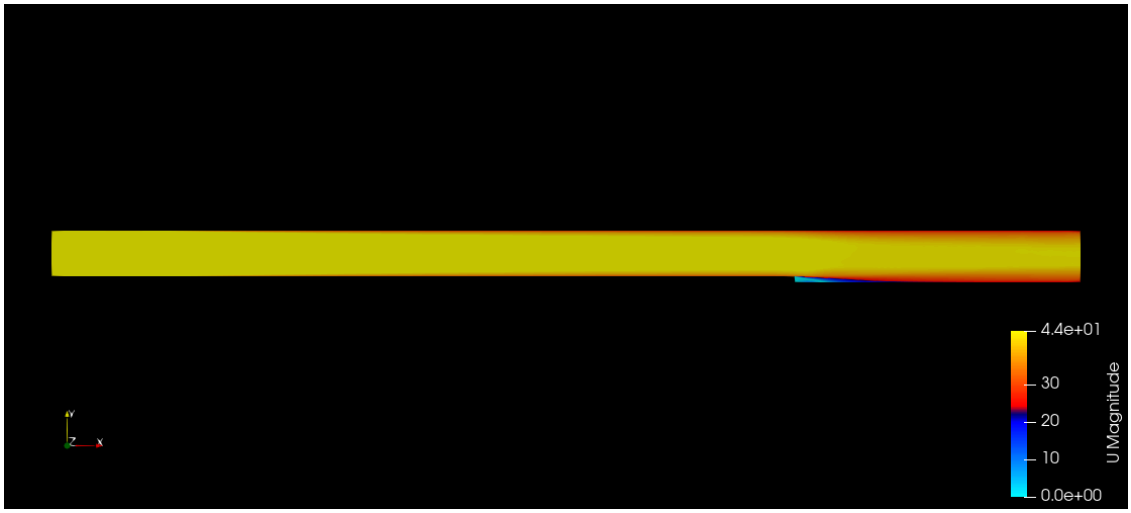


Figure 5: Velocity contour for best *base_OF* case

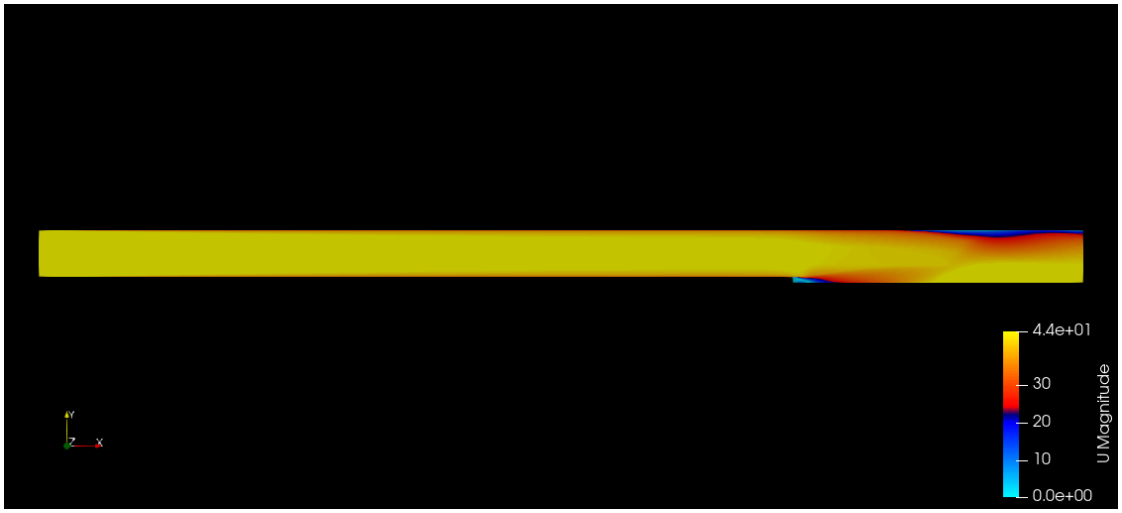


Figure 6: Velocity contour for the best NASA Grid case

Looking at the above figures, it is clear that the NASA case has not converged. We see some low velocity near the outlet (right end of the domain) that should be clarified through more iterations. Zooming in on the step region :

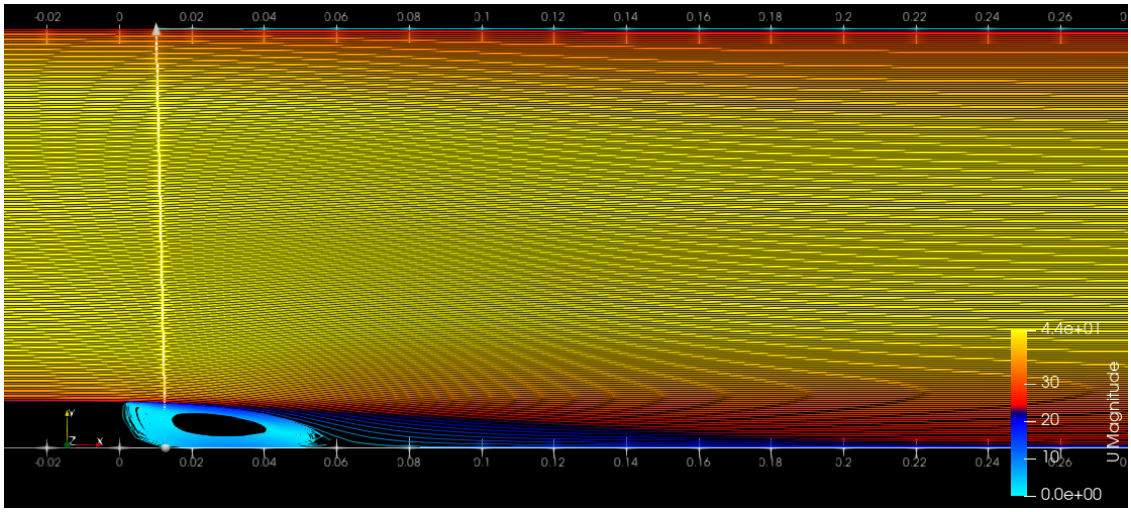


Figure 7: Velocity contour with streamlines near the step for best *base_OF* case

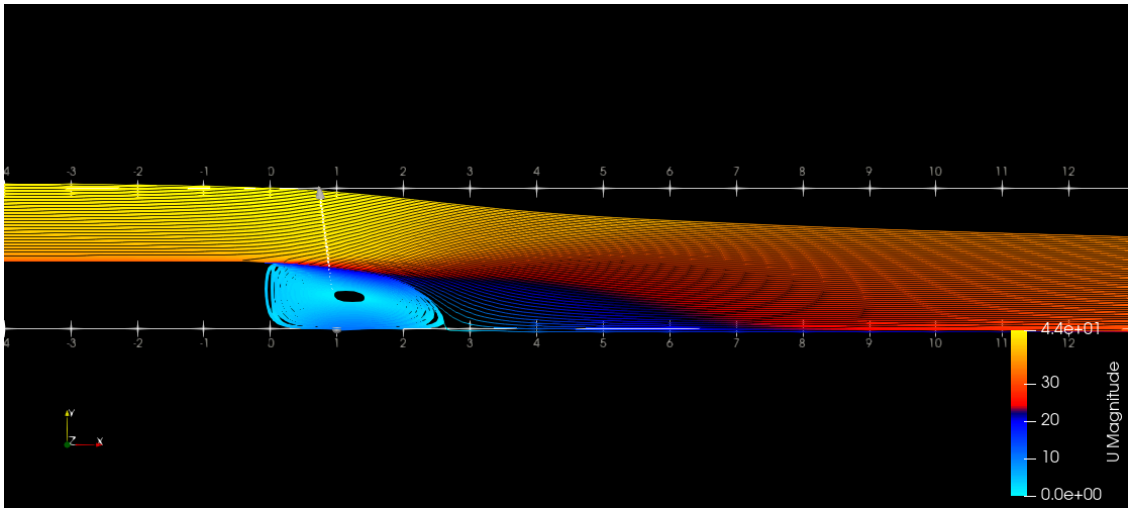


Figure 8: Velocity contour with streamlines near the step for the best NASA Grid case

From the NASA turbulence modelling resource, the expected reattachment point is at $\frac{x}{H} = 6.26 \pm 0.1$. For both cases, the reattachment region seems to be under predicted, with the *base_OF* case predicting 4.72 and the NASA grid around 3. If we make the assumption that the grid and setup are not at fault (this is a big assumption, but let's just make it for now), the difference in prediction is probably because of the changing turbulence coefficients. We are effectively meddling with the production of dissipation of turbulent kinetic energy, so it is expected to produce differences in regions of separated flow.

Additionally, because of the way these coefficients appear in the equations, I cannot confidently make any statements about any trends that might occur by increasing or decreasing these terms (i.e Other than saying that regions of flow separation will be affected, it's difficult to make any more concrete and detailed statements about the effects of the coefficients on the flow.).

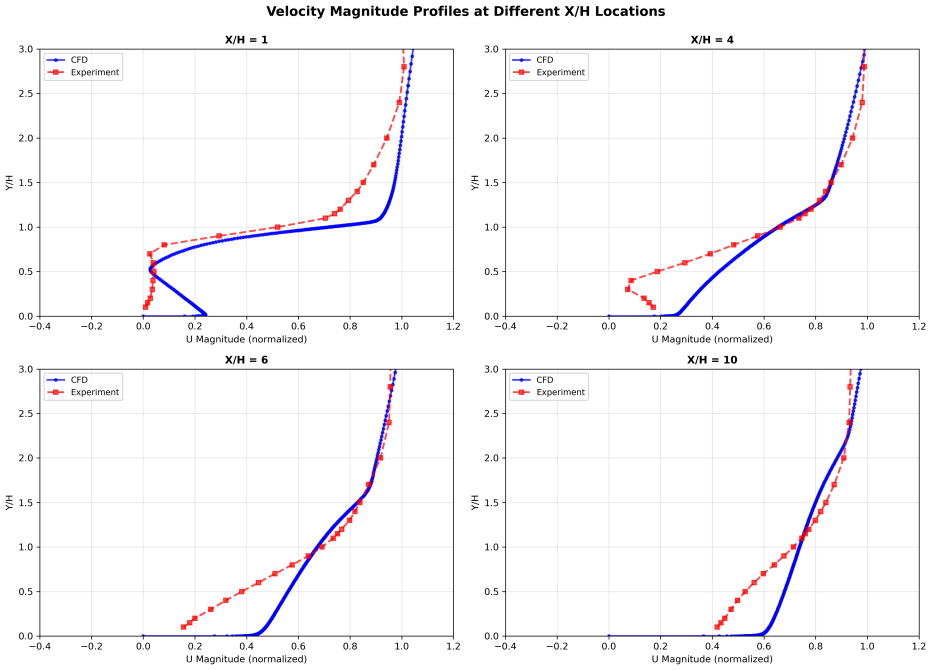


Figure 9: Velocity profiles for the best NASA Grid case

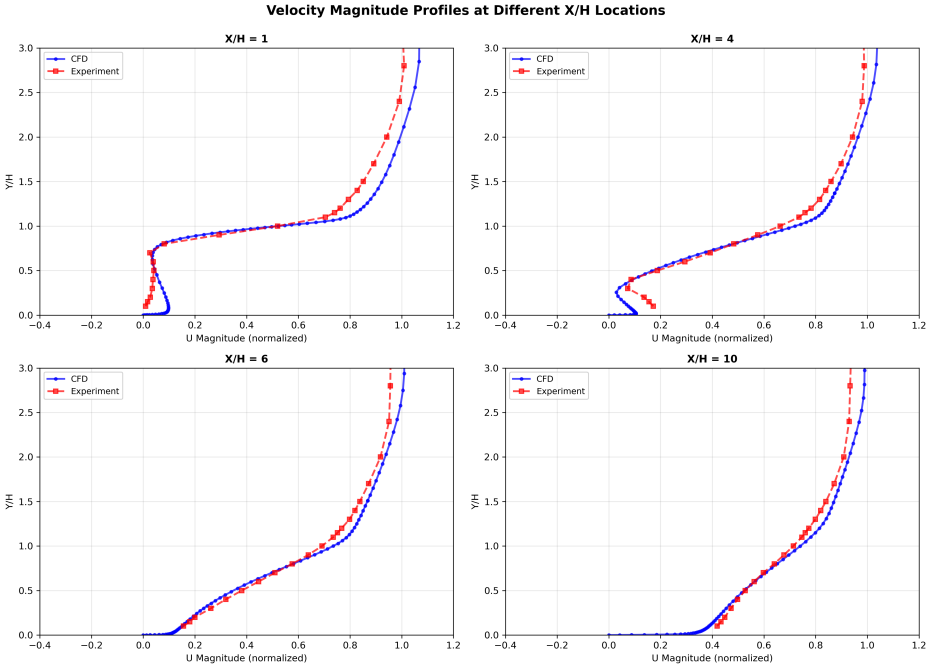


Figure 10: Velocity profiles for the best NASA Grid case

Once again, we see a better fit for the *base_OF* case velocity profiles. Look at Figure 9 x/H = 4, the experimental velocity profile suggests a recirculation zone exists (due to the initial negative velocity gradient), while the simulation has predicted a normal attached flow. The *base_OF* case has a much better fit for both the separated regions, and reattached regions.

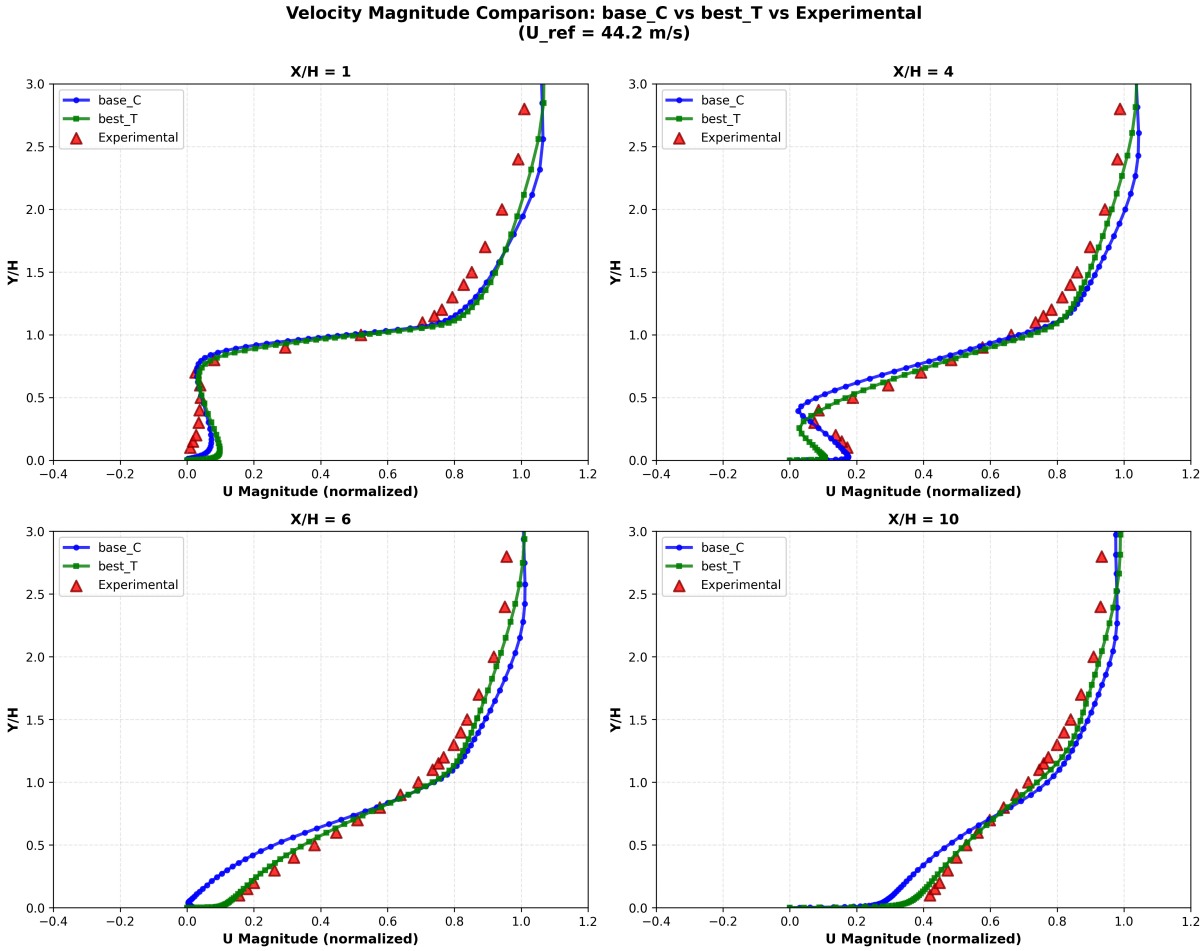


Figure 11: Velocity profiles for the best NASA Grid case

Figure 11 compares the velocity profiles of the best trial with the optimised coefficients (best_T), and base case with the original turbulence model coefficients (base_C) from the *base_OF* case. Looking at the subplots, we see that the optimisation has traded some fitness in the recirculation near wall region (x/H = 1,4) for better fit in the reattached region (x/H = 6, 10). This is the exact opposite I had expected, as I thought that by weighting the near wall region more, I'd get better fit in the recirculation zone.

But, in a way, I think this behavior is kind of expected. We see in x/H of 6, 10 that the near wall (up to y/H of 1.2) velocity profile fits very well to the experimental data. My guess is that the optimiser saw that by getting a better fit in x/H of 6,10 regions, it could reduce the error a lot, and so it might have focused on coefficients that facilitated this behaviour. Maybe if I had specifically assigned an error weight to the recirculation zone, I could have achieved a better fit in that region. Maybe next time I'll do that.

5. Summary

So, in the end, I got a 25% reduction in velocity error, and I think that's pretty good. For a learning exercise, I'd say this project went along pretty well. I learned about bayesian optimisation, its implementation in python, improved my coding skills, and also my problem solving and thinking ability in general.

Next I think I'll either try to apply this method to a different case (maybe an airfoil, or some other bluff body flow), or I may go in a completely different route and explore OpenFOAM in depth.

Thank you so much for reading. Please reach out to me on LinkedIn if you have any questions, or just want to chat about this.

Bibliography

[1] "Grids - 2D Backward Facing Step." Accessed: Jan. 11, 2026. [Online]. Available: https://turbmodels.larc.nasa.gov/backstep_grids.html

[2] R. Bai, J. Li, F. Zeng, and C. Yan, "Mechanism and Performance Differences between the SSG/LRR-ω and SST Turbulence Models in Separated Flows," *Aerospace*, vol. 9, no. 1, p. 20, Dec. 2021, doi: 10.3390/aerospace9010020.

[3] S. Younoussi and A. Ettaouil, "Calibration method of the k-ω SST turbulence model for wind turbine performance prediction near stall condition," *Heliyon*, vol. 10, no. 1, p. e24048, Jan. 2024, doi: 10.1016/j.heliyon.2024.e24048.

My bibliography is quite short because I haven't added every single source that I referred to. Maintaining a detailed bibliography would've taken quite a bit of effort for not much return on the learning objectives, so I didn't do it. Besides, this isn't meant to be an academic report anyways!